

I. INTRODUCTION

Ce chapitre vous introduira à l'apprentissage des différents opérateurs qui existent, tout en expliquant l'usage de chacun d'entre eux avec des exemples appropriés suivis des calculs, des comparaisons, des créations des groupes conditionnels, des recherches pour les expressions régulières avec les lignes des caractères (strings) et les méthodes de conversion des données.

Les opérateurs de MySQL peuvent être classifiés en quatre catégories différentes comme il est mentionné ci-dessous :

- ❖ Opérateurs arithmétiques
- ❖ Opérateurs de comparaisons
- ❖ Opérateur logiques
- ❖ Opérateurs binaires (bit).

OBJECTIFS DU COUR

Après avoir étudié ce cour vous serez à mesure de :

- Discuter l'importance et l'application des différents opérateurs.
- Décrire et appliquer les différents opérateurs comme les parenthèses, les comparaisons etc...
- Expliquer les contrôles des fonctions avec leurs applications
- Expliquer et appliquer les différentes fonctions de lignes de caractères (strings)
- Expliquer et appliquer les différents types d'opérateurs ainsi que les fonctions mathématiques
- Expliquer les fonctions DATE AND TIME (date et temps) avec les exemples appropriés.

II. LES OPERATEURS

1. Les parenthèses

(...)

Nous utilisons les parenthèses pour forcer l'ordre à laquelle une expression doit s'exécuter.

Considérons les exemples ci-dessous : $1+2*3$ sans parenthèse, on aura donc 7 et avec parenthèse cela donnera 9 comme c'est fait ci-dessous :

Exemple 1

```

c:\wamp_kips\bin\mysql\mysql5.5.8\bin\mysql.exe

mysql> select 1+2*3;
+-----+
| 1+2*3 |
+-----+
|      7 |
+-----+
1 row in set (0.04 sec)

mysql>

```

Exemple 2

```

c:\wamp_kips\bin\mysql\mysql5.5.8\bin\mysql.exe

mysql> select <1+2>*3;
+-----+
| <1+2>*3 |
+-----+
|      9 |
+-----+
1 row in set (0.00 sec)

mysql>

```

2. Operateurs de comparaisons

Lorsque vous travaillez avec la syntaxe MySQL SELECT, cela pourra vous donner envie de comparer le côté droit d'une expression au côté gauche de cette dernière et le résultat de ces comparaisons est de type TRUE (vrai), FALSE (faux) ou NULL (non applicable). Pour le faire, nous allons donc faire usage des différents opérateurs de comparaison qui existent comme l'indique le tableau ci-dessous :

opérateurs	fonctions
=	Egale a
◆ Aka !=	N'est pas égale a
⇔	Ce qui entraîne a / ce qui implique
<	Inférieur a
<=	Inférieur ou égale a
>	Supérieur a
>=	Supérieur ou égale a
Between	Ce qui existe dans une rangée donnée
IN	Ce qui existe dans un ensemble donnée
Is null	Ce qui est nul ou ensemble nul
Is not null	Ce qui n'est pas nul ou ensemble non nul
Like	Ce qui est comme, semblable a, identique a

```

mysql> select 6='6gfh', '6'='6gfh';
+-----+-----+
| 6='6gfh' | '6'='6gfh' |
+-----+-----+
|      1 |      0 |
+-----+-----+
1 row in set, 1 warning (0.32 sec)

mysql>

```

L'opérateur de comparaison peut aussi être utilisé pour comparer à la fois les nombres et l'ensemble de caractères (string ou mots). Prenant en compte que les nombres se comparent par rapport à leurs parties décimales et les mots par rapport aux différences qu'ils représentent sans pourtant prendre les cases de sensibilités (majuscule ou minuscule) en considération. L'opérateur d'égalité (=) est utilisé pour comparer les deux cotés d'une expression pour savoir s'ils étaient égaux. Comparons donc les nombres avec l'exemple ci-dessous :

```
mysql> select 2=2, 2.37=2.4, 2.425=2.4250000;
+-----+-----+-----+
| 2=2 | 2.37=2.4 | 2.425=2.4250000 |
+-----+-----+-----+
| 1 | 0 | 1 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Avec l'exemple ci-dessous, nous allons maintenant comparer les lignes de caractères ou les mots (strings) :

```
mysql> select 'h'='h', 'h'='B', 'B'='C', 'circle'='rond';
+-----+-----+-----+-----+
| 'h'='h' | 'h'='B' | 'B'='C' | 'circle'='rond' |
+-----+-----+-----+-----+
| 1 | 0 | 1 | 0 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

L'opposé de l'opérateur = est l'opérateur <>, qui est utilisé pour vérifier si les deux cotés d'une expression sont inégales ; le résultat retourne 1 si ils sont inégales et le contraire c'est-à-dire 0 s'ils sont égaux. Ce type compare aussi les nombres et les lignes de caractères.

Exemple 1 :

```
mysql> select 7<>7, 7<>8;
+-----+-----+
| 7<>7 | 7<>8 |
+-----+-----+
| 0 | 1 |
+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Exemple 2 :

```
mysql> select 'g'<>'G', 'g'<>'f', '7g'<>'7f';
+-----+-----+-----+
| 'g'<>'G' | 'g'<>'f' | '7g'<>'7f' |
+-----+-----+-----+
| 0 | 1 | 1 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Exemple 3 :

L'opérateur <> peut aussi être utilisé pour comparer les valeurs nulles :

```
c:\wamp_kips\bin\mysql\mysql5.5.8\bin\mysql.exe
mysql> select null <> null, 0<>null, 0<>0;
+-----+-----+-----+
| null <> null | 0<>null | 0<>0 |
+-----+-----+-----+
|          NULL |      NULL |    0 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

L'opérateur Between est utilisé pour vérifier si la valeur en vérification est comprise entre la rangée des valeurs spécifiées

```
mysql> select 10 between 0 and 100, 10 between 11 and 100;
+-----+-----+
| 10 between 0 and 100 | 10 between 11 and 100 |
+-----+-----+
|          1          |          0          |
+-----+-----+
1 row in set (0.30 sec)

mysql>
```

Nous pouvons aussi utiliser l'opérateur between pour aussi tester les caractères comme c'est l'illustre l'exemple ci-dessous :

```
mysql> select 'b' between 'a' and 'c', 'c' between 'a' and 'b';
+-----+-----+
| 'b' between 'a' and 'c' | 'c' between 'a' and 'b' |
+-----+-----+
|          1          |          0          |
+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Il est possible de renverser le résultat que produit l'opérateur between, en ajoutant « NOT » avant le mot between dans la syntaxe comme l'indique l'exemple ci-dessous :

```
mysql> select 'b' not between 'a' and 'c', 'c' not between 'a' and 'b';
+-----+-----+
| 'b' not between 'a' and 'c' | 'c' not between 'a' and 'b' |
+-----+-----+
|          0          |          1          |
+-----+-----+
1 row in set (0.00 sec)

mysql>
```

L'opérateur IN est utilisé pour vérifier si la valeur testée est comprise dans l'ensemble donné des valeurs qui ont été définies comme c'est fait ci-dessous :

```
mysql> select 7 IN (1,2,3,4,5,6,7,8,9);
+-----+
| 7 IN (1,2,3,4,5,6,7,8,9) |
+-----+
|          1          |
+-----+
1 row in set (0.25 sec)

mysql>
```

Ou encore :

```
mysql> select 'red' in ('yellow','red','orange');
+-----+
| 'red' in ('yellow','red','orange') |
+-----+
| 1 |
+-----+
1 row in set (0.00 sec)

mysql>
```

Pour tester la présence ou l'absence d'une valeur nulle dans une expression, on utilise les opérateurs IS NULL et IS NOT NULL qui sont créés spécialement pour gérer les comparaisons concernant les valeurs nulles.

Exemple :

```
c:\wamp_kips\bin\mysql\mysql5.5.8\bin\mysql.exe
mysql> select 88 is null, 0 is null, null is null, 1=null is null;
+-----+-----+-----+-----+
| 88 is null | 0 is null | null is null | 1=null is null |
+-----+-----+-----+-----+
| 0 | 0 | 1 | 1 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

L'opérateur $\Leftrightarrow$ (ce qu'implique) est un opérateur très spécial surtout quand il faut aller avec des valeurs null comme avec l'opérateur IS NULL. Avec cet opérateur, même dans le cas de déviation à la normale, MySQL retourne un résultat 1 (true) ou 0 (fausse) à propos de la comparaison même lorsque l'expression en question contient des valeurs Nulles :

```
mysql> select 2=null, 2<=>null, 4<=>1+3;
+-----+-----+-----+
| 2=null | 2<=>null | 4<=>1+3 |
+-----+-----+-----+
| NULL | 0 | 1 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> select null=null, null<=>null, 0=null, 0<=>null;
+-----+-----+-----+-----+
| null=null | null<=>null | 0=null | 0<=>null |
+-----+-----+-----+-----+
| NULL | 1 | NULL | 0 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Il y a des moments lors de vos nombreuses balades sur la plateforme MySQL, il se peut vous ayez envi de faire un système de recherche des données dans votre base, alors dans ce cas, l'opérateur LIKE vous un travail parfait. L'opérateur LIKE fonctionne de cette manière : il sélectionne et affiche toutes valeurs qui correspondent à celle que l'on venait

d'entrer (pour la recherche) et celles qui possèdent dans leurs composition, une partie qui est exactement identique au mot entré. Considérons donc les exemples ci-dessous :

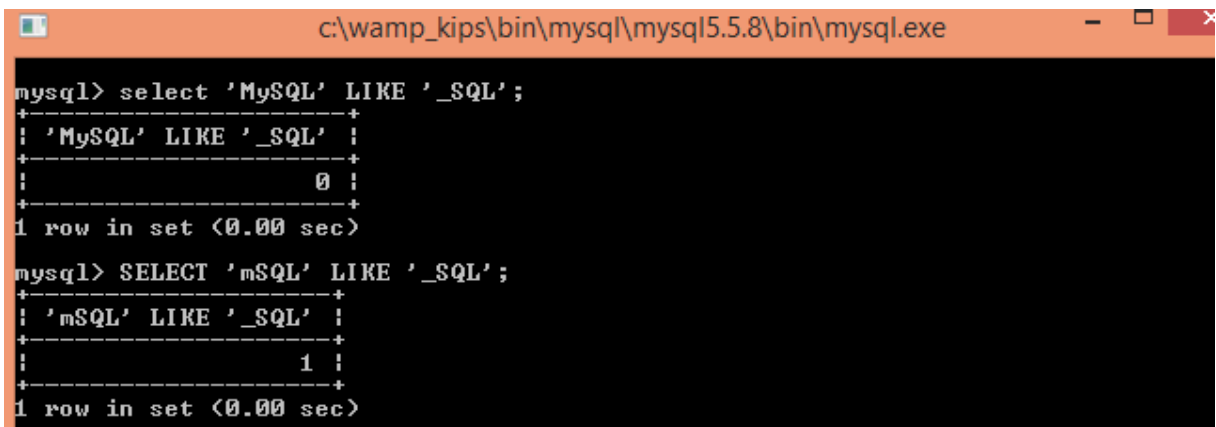
```
mysql> select * from services;
+----+-----+-----+-----+
| sid | snom          | bid  | branches |
+----+-----+-----+-----+
| 1   | recrutement   | 1011 | rh        |
| 2   | gestion de donnees | 1030 | informatique |
| 3   | administration | 1031 | executif  |
| 4   | depanage      | 1031 | maintenance |
| 5   | reabilitation | 1031 | maintenance |
+----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> select snom like '%dex%' from services;
+-----+
| snom like '%dex%' |
+-----+
| 0                  |
| 1                  |
| 0                  |
| 1                  |
| 0                  |
+-----+
5 rows in set (0.00 sec)

mysql>
```

On constate que dans cet exemple partout dans le tableau ou l'on peut avoir la combinaison 'de', MySQL affiche vrai (1) et le contraire (0).

Nous pouvons aussi marquer un seul caractère au lieu de toute une ligne de caractères (string) une utilisant la barre de 8 (_) comme c'est fait ci-dessous :



```
c:\wamp_kips\bin\mysql\mysql5.5.8\bin\mysql.exe

mysql> select 'MySQL' LIKE '_SQL';
+-----+
| 'MySQL' LIKE '_SQL' |
+-----+
| 0                  |
+-----+
1 row in set (0.00 sec)

mysql> SELECT 'mSQL' LIKE '_SQL';
+-----+
| 'mSQL' LIKE '_SQL' |
+-----+
| 1                  |
+-----+
1 row in set (0.00 sec)
```

Et en suite :

```

c:\wamp_kips\bin\mysql\mysql5.5.8\bin\mysql.exe
mysql>
mysql> SELECT '_SQL' LIKE '_SQL';
+-----+
| '_SQL' LIKE '_SQL' |
+-----+
| 1 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT 'KSQL' LIKE '_SQL';
+-----+
| 'KSQL' LIKE '_SQL' |
+-----+
| 1 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT 'kSQL' LIKE '_SQL';
+-----+
| 'kSQL' LIKE '_SQL' |
+-----+
| 1 |
+-----+
1 row in set (0.00 sec)

```

III. OPERATEURS LOGIQUES

MySQL Tout comme la plupart des langages de programmation met à notre disposition quatre opérateurs logiques capables de tester la validité d'une ou plusieurs expressions (ou ensemble d'expressions). Le résultat produit par ces opérateurs sont souvent 1 (vrai), 0 (faux) et nul (ne pouvons être déterminé).

Le tableau ci-dessous liste tous les opérateurs logiques que dispose MySQL :

Opérateurs	Fonctions ou significations
NOT aka !	N'est pas, le contraire d'est la logique « n'est pas ».
AND aka &&	La logique « et »
OR aka	La logique « ou »
XOR	La logique exclusive d'où « exclusif ou »

1. OPERATEUR NOT :

Cet opérateur renverse le sens des résultats logique des tests ou expressions dans lesquels il est inclut tout en changeant le vrai en faux et le faux en vrai.

Exemple :

```

mysql> select not 1, not 0, not(2=2),not(100>200);
+-----+-----+-----+-----+
| not 1 | not 0 | not(2=2) | not(100>200) |
+-----+-----+-----+-----+
| 0 | 1 | 0 | 1 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>

```

2. OPERATEUR AND :

Cet opérateur nous donne la possibilité de tester la validité de deux ou plusieurs valeurs. Il retourne un résultat vrai si tous les composants issus du test sont vrais et non nuls ; et le contraire produit un faux.

Exemple :

```
mysql> select 2=2 and 900<100, 'a'='a' and 'c'<'d';
+-----+-----+
| 2=2 and 900<100 | 'a'='a' and 'c'<'d' |
+-----+-----+
|          0      |          1          |
+-----+-----+
1 row in set (0.00 sec)

mysql>
```

3. OPERATEUR OR :

Cet opérateur retourne vrai si une expression au moins dans le test est vraie non nulle et produit un faux si les deux composants d'une expression sont fausses.

Exemple :

```
mysql> select 2=2 or 900<100, 'a'='a' or 'c'<'d';
+-----+-----+
| 2=2 or 900<100 | 'a'='a' or 'c'<'d' |
+-----+-----+
|          1      |          1          |
+-----+-----+
1 row in set (0.00 sec)

mysql>
```

4. OPERATEUR XOR (exclusif OR):

Cet opérateur retourne un vrai si soit un des composants est vrai mais pas les deux.

Par exemple :

```
mysql> select 1=1 xor 2= 4, 1<2 xor 9<10;
+-----+-----+
| 1=1 xor 2= 4 | 1<2 xor 9<10 |
+-----+-----+
|          1      |          0          |
+-----+-----+
1 row in set (0.00 sec)

mysql>
```

IV. OPERATEURS BIT (à petites expressions)

MySQL possède des opérateurs créés spécialement pour des petites expressions qui sont très manipulables et rapides. Le tableau ci-dessous énumère tous les opérateurs BIT que dispose MySQL.

Operateurs	Fonctions et significations
&	Bitwise AND
	Bitwise OR
^	Bitwise XOR (exclusif OR)
>>	Bitwise decalage a droite (right shift).
<<	Bitwise decalage a gauche (left shift).

1. OPERATEURS | et &:

L'opérateur | est utilisé pour réaliser des opérations bitwise OR des bits alors que l'opérateur & est utilisé pour réaliser les opérations bitwise AND des bits.

Exemple :

```

c:\wamp_kips\bin\mysql\mysql5.5.8\bin\mysql.exe

mysql> select 16 | 32, 9 | 4;
+-----+-----+
| 16 | 32 | 9 | 4 |
+-----+-----+
|    48 |    13 |
+-----+-----+
1 row in set (0.00 sec)

mysql> select 30 & 10, 8 & 16;
+-----+-----+
| 30 & 10 | 8 & 16 |
+-----+-----+
|    10 |    0 |
+-----+-----+
1 row in set (0.00 sec)

mysql>

```

2. OPERATEURS << et >>:

Il est très possible de décaler les bits vers la gauche ou vers la droite en utilisant les opérateurs << et >> respectivement.

Exemple :

```

mysql> select 1<<7, 64>>1;
+-----+-----+
| 1<<7 | 64>>1 |
+-----+-----+
| 128 |    32 |
+-----+-----+
1 row in set (0.30 sec)

mysql> select 1<<0, 0>>1;
+-----+-----+
| 1<<0 | 0>>1 |
+-----+-----+
|    1 |    0 |
+-----+-----+
1 row in set (0.00 sec)

mysql>

```

3. OPERATEUR ^:

L'opérateur ^ réalise les bitwise XOR des bits.

Exemple :

```
c:\wamp_kips\bin\mysql\mysql5.5.8\bin\mysql.exe
mysql> select 1^0, 0^1, 17^9, 143^66;
+-----+
| 1^0 | 0^1 | 17^9 | 143^66 |
+-----+
| 1 | 1 | 24 | 205 |
+-----+
1 row in set (0.00 sec)
mysql>
```

V. CONTROLES FLOW DE LA FONCTION IF

1. La fonction IF

La fonction IF est de la forme : IF (EXPR1, EXPR2, EXPR3). Si l'expression 1 est vraie, alors la fonction IF retournera l'expression 2 comme résultat. Et si l'expression 1 est fausse alors la fonction IF retournera l'expression 3 comme résultat.

Exemple :

```
mysql> select if(1>2,2,3);
+-----+
| if(1>2,2,3) |
+-----+
| 3 |
+-----+
1 row in set (0.30 sec)
mysql>
```

Exemple 2 :

```
mysql> select if(strcmp('test','test1'),'no','yes');
+-----+
| if(strcmp('test','test1'),'no','yes') |
+-----+
| no |
+-----+
1 row in set (0.31 sec)
mysql>
```

2. La fonction IFNULL.

La fonction IFNULL est de la forme IFNULL (EXPR1, EXPR2). Ici nous utiliserons que deux expressions au lieu de trois entre parenthèses avec la fonction IFNULL ; cette fonction retourne l'expression 1 si elle n'est pas nulle mais au cas contraire, elle retournera l'expression 2 comme l'indique l'exemple ci-dessous :

```
mysql> select ifnull(1,0);
+-----+
| ifnull(1,0) |
+-----+
|          1 |
+-----+
1 row in set (0.00 sec)

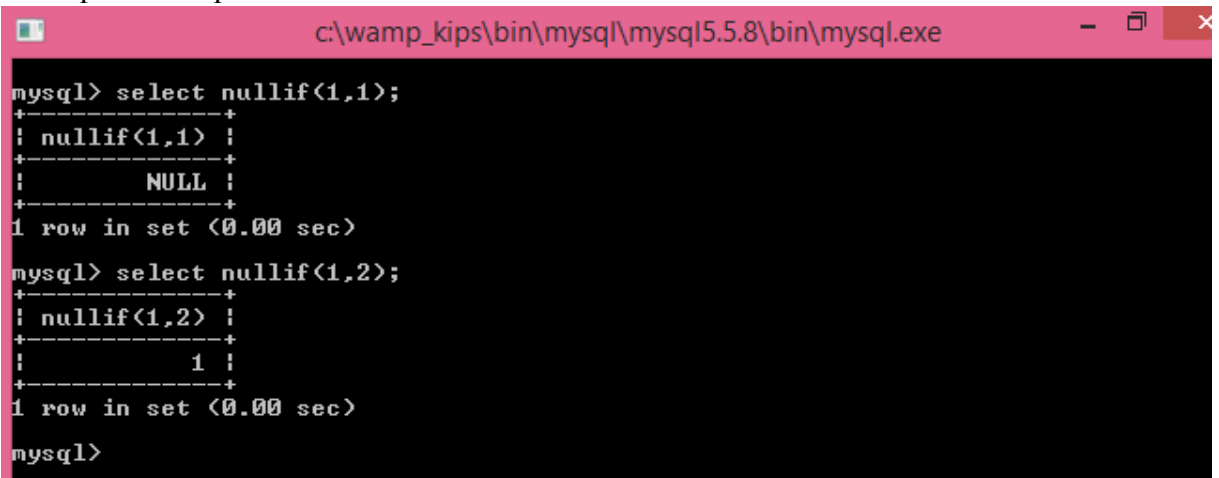
mysql> select ifnull(null,10);
+-----+
| ifnull(null,10) |
+-----+
|          10 |
+-----+
1 row in set (0.00 sec)

mysql> select ifnull(1/0,'yes');
+-----+
| ifnull(1/0,'yes') |
+-----+
| yes               |
+-----+
1 row in set (0.30 sec)

mysql>
```

3. La fonction NULLIF

La fonction NULLIF est de la forme NULLIF (EXPR1, EXPR2). Cette fonction retourne un résultat nul lorsque l'expression1 et l'expression2 sont égales comme l'indique l'exemple ci-dessous :



```
c:\wamp_kips\bin\mysql\mysql5.5.8\bin\mysql.exe

mysql> select nullif(1,1);
+-----+
| nullif(1,1) |
+-----+
|          NULL |
+-----+
1 row in set (0.00 sec)

mysql> select nullif(1,2);
+-----+
| nullif(1,2) |
+-----+
|          1 |
+-----+
1 row in set (0.00 sec)

mysql>
```

VI. LES FONCTIONS STRING (CARACTERE ET CHAINE DE CARACTERES)

Il existe plusieurs types des fonctions STRING qui nous permettent de faire énormément de choses selon nos envies. Ci-dessous sont quelques-unes entre elle :

1. ASCII (str)

Elle retourne la valeur numérique du caractère se trouvant à gauche d'un string (mot). Et elle travaille avec les caractères possédant les valeurs numériques allant de 0 à 255. Exemple :

```
c:\wamp_kips\bin\mysql\mysql5.5.8\bin\mysql.exe

mysql> select ASCII('2');
+-----+
| ASCII('2') |
+-----+
|          50 |
+-----+
1 row in set (0.30 sec)

mysql> select ASCII(2);
+-----+
| ASCII(2) |
+-----+
|          50 |
+-----+
1 row in set (0.00 sec)

mysql> select ASCII('dx');
+-----+
| ASCII('dx') |
+-----+
|          100 |
+-----+
1 row in set (0.00 sec)

mysql>
```

2. BIN(N)

Cette fonction retourne la représentation string d'une valeur binaire. Cela marche parfaitement avec les nombres que les mots.

Exemple :

```
mysql> select BIN(12);
+-----+
| BIN(12) |
+-----+
| 1100    |
+-----+
1 row in set (0.25 sec)
```

3. BIT_LENGTH(str)

Elle retourne la longueur d'un mot (string) en bit.

Exemple :

```
c:\wamp_kips\bin\mysql\mysql5.5.8\bin\mysql.exe

mysql> select bit_length('text');
+-----+
| bit_length('text') |
+-----+
|          32 |
+-----+
1 row in set (0.00 sec)

mysql> select bit_length('kipsmav');
+-----+
| bit_length('kipsmav') |
+-----+
|          56 |
+-----+
1 row in set (0.00 sec)

mysql>
```

4. CHAR(N...)

Cette fonction interprète l'argument étant des nombres et retourne la correspondance en ligne de caractère (mots ou string).

Exemple :

```
c:\wamp_kips\bin\mysql\mysql5.5.8\bin\mysql.exe

mysql> select char <77,121,83,81,'76'>;
+-----+
| char <77,121,83,81,'76'> |
+-----+
| MySQL                    |
+-----+
1 row in set <0.00 sec>

mysql> select char <77,77.3,'77.3'>;
+-----+
| char <77,77.3,'77.3'> |
+-----+
| MMM                  |
+-----+
1 row in set, 1 warning <0.00 sec>

mysql>
```

5. CHAR_LENGTH (str)

Cette fonction retourne la longueur d'une chaîne de caractères

Exemple :

```
mysql> select char_length('kipsmau');
+-----+
| char_length('kipsmau') |
+-----+
| 7                       |
+-----+
1 row in set <0.00 sec>

mysql>
```

6. CONCAT (str1, str2,...)

Cette fonction retourne l'assemblage de tous les arguments proposés dès le départ.

Exemple :

```
c:\wamp_kips\bin\mysql\mysql5.5.8\bin\mysql.exe

mysql> select concat <'My','S','QL'>;
+-----+
| concat <'My','S','QL'> |
+-----+
| MySQL                  |
+-----+
1 row in set <0.00 sec>

mysql>
```

7. INSERT (str, pos, Len, newstr)

Elle retourne le string str avec un sous-string commençant à une position POS à la longueur LEN et qui est remplacé par le string NEWSTR.

Exemple :

```
c:\wamp_kips\bin\mysql\mysql5.5.8\bin\mysql.exe
mysql> SELECT INSERT<'QUADRATIC',3,4,'WHAT'>;
+-----+
| INSERT<'QUADRATIC',3,4,'WHAT'> |
+-----+
| QUWHATTIC                        |
+-----+
1 row in set (0.00 sec)

mysql>
```

8. LPAD (str, len, padstr)

Cette fonction retourne la chaîne de caractères str, par rapport à la longueur qui a été fixée. Au cas où le string possède plus de caractères que la longueur établie, la fonction retourne que les caractères satisfaisant la condition. Cette fonction travaille sur les caractères à gauche du string en question.

Exemple :

```
+-----+
| LPAD <'hi',4,'??'> |
+-----+
| ??hi                |
+-----+
1 row in set (0.01 sec)

mysql> select LPAD <'hi',1,'??'>;
+-----+
| LPAD <'hi',1,'??'> |
+-----+
| h                  |
+-----+
1 row in set (0.00 sec)

mysql> select LPAD <'hi',3,'??'>;
+-----+
| LPAD <'hi',3,'??'> |
+-----+
| ?hi                |
+-----+
1 row in set (0.00 sec)

mysql>
```

9. LTRIM (str)

Cette fonction retourne une chaîne de caractères tout en éliminant les espaces présentes aux extrémités de la chaîne c'est-à-dire au commencement et à la fin de la chaîne.

Exemple :

```
mysql> select LTRIM<' BARBAR'>;
+-----+
| LTRIM<' BARBAR'> |
+-----+
| BARBAR           |
+-----+
1 row in set (0.31 sec)

mysql> select LTRIM<'1 BARBAR'>;
+-----+
| LTRIM<'1 BARBAR'> |
+-----+
| 1 BARBAR         |
+-----+
1 row in set (0.00 sec)

mysql>
```

10. REPEAT (str, count)

Cette fonction donne en retour la répétition de la chaîne de caractères le nombre de fois qu'il a été indiqué.

Exemple :

```
mysql> SELECT REPEAT('MySQL',3);
+-----+
| REPEAT('MySQL',3) |
+-----+
| MySQLMySQLMySQL   |
+-----+
1 row in set (0.00 sec)

mysql>
```

11. REPLACE (str, de_str, a_str)

Cette fonction retourne une chaîne de caractères avec toutes les occurrences de ce string par rapport au string 'de_str' qui sera remplacé par le string 'a_str' comme l'indique l'exemple ci-dessous :

```
c:\wamp_kips\bin\mysql\mysql5.5.8\bin\mysql.exe
+-----+
| row in set (0.00 sec)
mysql> SELECT REPLACE('www.mysql.com','w','W');
+-----+
| REPLACE('www.mysql.com','w','W') |
+-----+
| WWWWWW.mysql.com                 |
+-----+
1 row in set (0.00 sec)

mysql>
```

NB : il existe plusieurs fonctions des chaînes de caractères (string) mais nous avons juste pris quelques-unes d'entre elles.

VII. LES FONCTIONS NUMERIQUES

1. OPERATEURS ARITHMETIQUES

Le tableau ci-dessous énumère les différents opérateurs numériques qui existent :

NOMS	DESCRIPTION
/	Opérateur de division
-	Opérateur de soustraction
%	Opérateur modulo
+	Opérateur d'addition
*	Opérateur de multiplication.

Exemple :

```

mysql> select 4/2;
+-----+
| 4/2 |
+-----+
| 2.0000 |
+-----+
1 row in set (0.00 sec)

mysql> select 4-2;
+-----+
| 4-2 |
+-----+
| 2 |
+-----+
1 row in set (0.29 sec)

mysql> select 4%2;
+-----+
| 4%2 |
+-----+
| 0 |
+-----+
1 row in set (0.00 sec)

mysql> select 4+2;
+-----+
| 4+2 |
+-----+
| 6 |
+-----+
1 row in set (0.00 sec)

mysql> select 4*2;
+-----+
| 4*2 |
+-----+
| 8 |
+-----+
1 row in set (0.00 sec)

mysql>

```

2. FONCTIONS MATHÉMATIQUES

Le ci-dessous énumère les fonctions mathématiques que disposent MySQL :

NOMS	DESCRIPTIONS
ABS ()	Valeurs absolues
ACOS ()	Arc de cosinus
ASIN ()	Arc de sinus
ATAN2 (), ATAN ()	Arc de tangente de deux arguments
ATAN ()	Arc de tangent
CEIL ()	Le plus petit nombre pas < a l'argument
CEILING ()	Le plus petit nombre pas < a l'argument
CONV ()	Convertir les nombres en d'autres bases
COS ()	Fonction cosinus
COT ()	Fonction cotangente
DEGREES ()	Fonction pour convertir le radian en degré
EXP ()	Elevé a la puissance (exposant de)
FLOOR ()	Le plus grand nombre pas > a l'argument
LN ()	Fonction pour le logarithme d'un argument
LOG 10 ()	Fonction logarithme a la base 10 d'un argument
LOG 2 ()	Fonction logarithme a la base 2 d'un argument
LOG ()	Fonction pour le logarithme du premier argument.
MOD ()	la fonction modulo
OCT ()	Fonction pour convertir les nombre de la base décimale (10) à la base octal (8).
PI ()	Retourne la valeur de pi
POW ()	Retourne les arguments élevés à la puissance spécifiée.
POWER ()	Retourne les arguments élevés à la puissance spécifiée.
RADIANS ()	Retourne l'argument converti en radian
SIN ()	Retourne le sinus d'un argument
SQRT ()	Retourne la racine carrée d'un argument
TAN ()	Retourne la tangente d'un argument
TRUNCATE ()	Retourne la valeur tronquée d'un nombre ou une ligne de caractères.

Exemples :

1. La fonction absolue

```

+-----+
| ABS(2) |
+-----+
|      2 |
+-----+
1 row in set (0.31 sec)

mysql> select ABS(-32);
+-----+
| ABS(-32) |
+-----+
|      32 |
+-----+
1 row in set (0.00 sec)

mysql>

```

2. La fonction arc de cosinus

```
mysql> select ACOS(1);
+-----+
| ACOS(1) |
+-----+
| 0       |
+-----+
1 row in set (0.32 sec)

mysql> select ACOS(1.0001);
+-----+
| ACOS(1.0001) |
+-----+
| NULL         |
+-----+
1 row in set (0.00 sec)

mysql> select ACOS(0);
+-----+
| ACOS(0)      |
+-----+
| 1.5707963267948966 |
+-----+
1 row in set (0.00 sec)

mysql>
```

VIII. LES FONCTIONS DE DATE ET DE TEMPS

La section ci, décrit les fonctions qui permettent de manipuler les valeurs temporaires. Il en existe plusieurs de ces types de fonctions et le tableau ci-dessous regorge quelques-unes entre elles :

NOMS	DESCRIPTIONS
ADDDATE ()	Ajouter les dates
ADDTIME ()	Ajouter le temps
CONVERT_TZ ()	Convertir d'un fuseau horaire à un autre
CURDATE ()	Retourne la date actuelle
CURRENT_DATE ()	Retourne la date actuelle
CURRENT_TIME ()	Retourne le temps actuel
CURTIME ()	Retourne le temps actuel
DATE_ADD ()	Ajouter deux dates
DATE_FORMAT ()	Le format de la date
DATE_SUB ()	Soustraire deux dates
DAY ()	Le jour du mois (1 ^{er} , 23, 5,...)
DAYNAME ()	Le jour de la semaine (lundi, mardi,...)
FROM_DAYS ()	Convertit le numéro d'un jour en date
HOUR ()	Extrait l'heur
LAST_DAY ()	Retourne le dernier jour d'un mois
MINUTE ()	Retourne les minutes d'un argument
MONTH ()	Retourne les mois d'une date passée
SEC_TO_TIME ()	Convertit les secondes en hh :mm :ss
TIME_TO_SEC ()	Retourne les arguments convertis en secondes

Exemple : considérons l'exemple ci-dessous pour plus d'illustration :

```
mysql> select DAYOFMONTH ('2001-11-00'), MONTH('2005-00-00');
+-----+-----+
| DAYOFMONTH ('2001-11-00') | MONTH('2005-00-00') |
+-----+-----+
| 0 | 0 |
+-----+-----+
1 row in set (0.33 sec)

mysql> select DAYOFMONTH ('2001-11-03'), MONTH('2005-05-04');
+-----+-----+
| DAYOFMONTH ('2001-11-03') | MONTH('2005-05-04') |
+-----+-----+
| 3 | 5 |
+-----+-----+
1 row in set (0.00 sec)

mysql>
```

IX. RESUME

Ce chapitre a eu à parcourir les titres ci-dessous :

- ❖ Les operateurs
- ❖ Le contrôle flow des fonctions
- ❖ Les fonctions des chaines de caractères
- ❖ Les fonctions numériques
- ❖ Les fonctions des dates et des temps

X. QUESTIONS TERMINALES

Ci-dessous sont les questions terminales R7

- ❖ Définissez les termes suivants :
 1. Parenthèses
 2. Operateurs de comparaison
 3. Operateurs logiques
- ❖ Discuter à propos de n'importe qu'elle 5 fonctions des chaines de caractères